Phase 1: Course and Lecture Preparation and Delivery Support

The objective of this phase is to build an AI co-professor that helps a professor prepare and adapt a course or individual lecture using course objectives, available class time, expected student level, and relevant course material.

1. The exact features

Course and lecture context: The professor will provide information such as the syllabus, course objectives, topic, expected student level, available class time, previous lecture information, and relevant course material.

Structured lecture planning: The system will generate a structured lecture plan, including the concepts to cover, their sequence, and approximate time allocation.

Teaching support: The assistant will suggest real-world and professional examples, industry applications, classroom activities, and interactive ways of presenting a topic.

Additional explanations: The professor will be able to ask for alternative explanations, examples, or ways of presenting difficult concepts through the chat interface.

Verified resources: When external resources are recommended, the system will check that they exist and are valid before presenting them as verified resources. It should not invent or assume resources.

Two-sided interaction: The system will identify important missing information and ask the professor for clarification or additional resources instead of silently making assumptions.

Lecture re-calibration: The professor will be able to update the course schedule when circumstances change, such as missed lectures, holidays, or changes in available class time. The system will recalculate the remaining schedule accordingly.

Professor control: The professor will be able to review, edit, accept, reject, or regenerate generated plans, examples, activities, and recommendations.

2. Timeline

Week 1: 14 Sept - 20 Sept

- Finalize the requirements for Phase 1.
- Define the information required from the professor for course and lecture preparation.

- Define the structure of the generated lecture plan.
- Define how the system will identify missing information and ask for clarification.
- Design the overall website and ChatGPT/Gemini-like chat interface.

Week 2: 21 Sept - 27 Sept

- Implement course and lecture context input.
- Implement upload and retrieval of course materials.
- Build the first version of lecture-plan generation.
- Generate concept sequences and approximate time allocation.
- Allow the professor to provide and update lecture-specific information.

Week 3: 28 Sept - 4 Oct

- Implement teaching support features.
- Generate real-world and professional examples and industry applications.
- Suggest classroom activities and interactive ways of presenting topics.
- Generate possible misconceptions and additional explanations for difficult concepts.
- Integrate these capabilities into the professor chat interface.

Week 4: 5 Oct - 11 Oct

- Implement the resource recommendation and verification workflow.
- Ensure recommended external resources can be checked for existence and validity.
- Implement two-sided interaction for missing information and clarification.
- Implement professor review, editing, and regeneration of generated content.
- Integrate these workflows into the main chat experience.

Week 5: 12 Oct - 18 Oct

- Implement lecture and course schedule re-calibration.
- Handle missed lectures, holidays, and changes in available class time.
- Recalculate remaining topics, lecture sequence, and time allocation after schedule changes.
- Integrate re-calibration with the existing lecture planning workflow.
- Complete integration of all major Phase 1 features into one working system.

Week 6: 19 Oct - 25 Oct

- Perform comprehensive testing of all Phase 1 features.
- Test the system with different course, lecture, duration, and student-level scenarios.
- Test resource verification, missing-information handling, and schedule re-calibration.
- Identify and fix bugs and inconsistencies.
- Improve reliability, usability, and response quality based on testing.
- Prepare the final Phase 1 demonstration.

3. Outcomes

- A working website with a ChatGPT/Gemini-like chat interface designed for professors.
- The ability to generate structured lecture plans using course and lecture context.
- Concept sequencing and approximate time allocation for a lecture.
- Relevant teaching examples, professional and industry applications, classroom activities, and interactive teaching suggestions.
- Possible misconceptions and additional explanations for difficult concepts.
- The ability to identify missing information and ask the professor for clarification.
- External resource recommendations that are not invented or assumed.
- A resource verification process that checks recommended resources before they are presented as verified.
- The ability to re-calibrate an existing course or lecture schedule when the professor's availability or schedule changes.
- Professor control over reviewing, modifying, accepting, rejecting, and regenerating generated content.
- All major Phase 1 capabilities integrated into a single usable assistant.

4. Tests verifying the outcomes

Lecture plan generation test: Provide a course, topic, student level, and lecture duration. Verify that the system produces a structured and relevant lecture plan that fits the available time.

Concept sequencing test: Provide a set of concepts for a lecture. Verify that the generated sequence is coherent and suitable for the topic.

Time allocation test: Generate plans for different lecture durations. Verify that the time allocation changes appropriately and remains within the available duration.

Teaching support test: Verify that examples, professional applications, classroom activities, interactive teaching suggestions, and possible misconceptions are relevant to the topic.

Course-context test: Provide course-specific material and verify that the generated plan and suggestions use the provided material where relevant rather than relying only on generic information.

Missing-information test: Intentionally leave important information unspecified. Verify that the assistant asks for clarification instead of silently making unsupported assumptions.

Resource verification test: Provide scenarios involving valid and non-existent resources. Verify that the system does not present non-existent resources as valid or verified.

Re-calibration test: Create an initial course schedule and then introduce a missed lecture or holiday. Verify that the remaining schedule, topics, and time allocation are recalculated correctly.

Professor editing test: Modify or reject part of a generated plan and ask the system to regenerate it. Verify that the updated output reflects the professor's changes.

Two-sided interaction test: Give the assistant incomplete information and verify that it identifies the missing information and asks an appropriate question before producing a final recommendation.

Chat continuity test: Conduct a multi-step conversation about a lecture. Verify that the assistant retains and uses the relevant context from earlier messages.

End-to-end test: Start with a new course, prepare a lecture, modify its duration, request teaching examples, introduce a schedule disruption, and perform re-calibration through the same interface.

Proposed Tech Stack

Frontend: React with TypeScript and Tailwind CSS for the professor dashboard and chat interface.

Backend: FastAPI for APIs and application logic.

LLM: Gemini, OpenAI, or Claude for lecture planning, explanations, examples, and recommendations.

Course Material: Retrieval-Augmented Generation (RAG) for using uploaded syllabus, notes, and course material as context.

Vector Database: ChromaDB or Qdrant for storing and retrieving relevant course material.

Application Database: PostgreSQL for application data and course/lecture information.

AI Workflow: A modular workflow for planning, clarification, verification, and re-calibration, with LangGraph as an optional implementation.

Resource Verification: A web/resource checking layer to verify externally recommended resources.